PROJETO CLIMAALERTA

Desenvolvendo o ClimaAlerta passo a passo no PyCharm

Python, dashboards, Selenium, APIs, Pandas, banco de dados, testes, depuração, Git e documentação

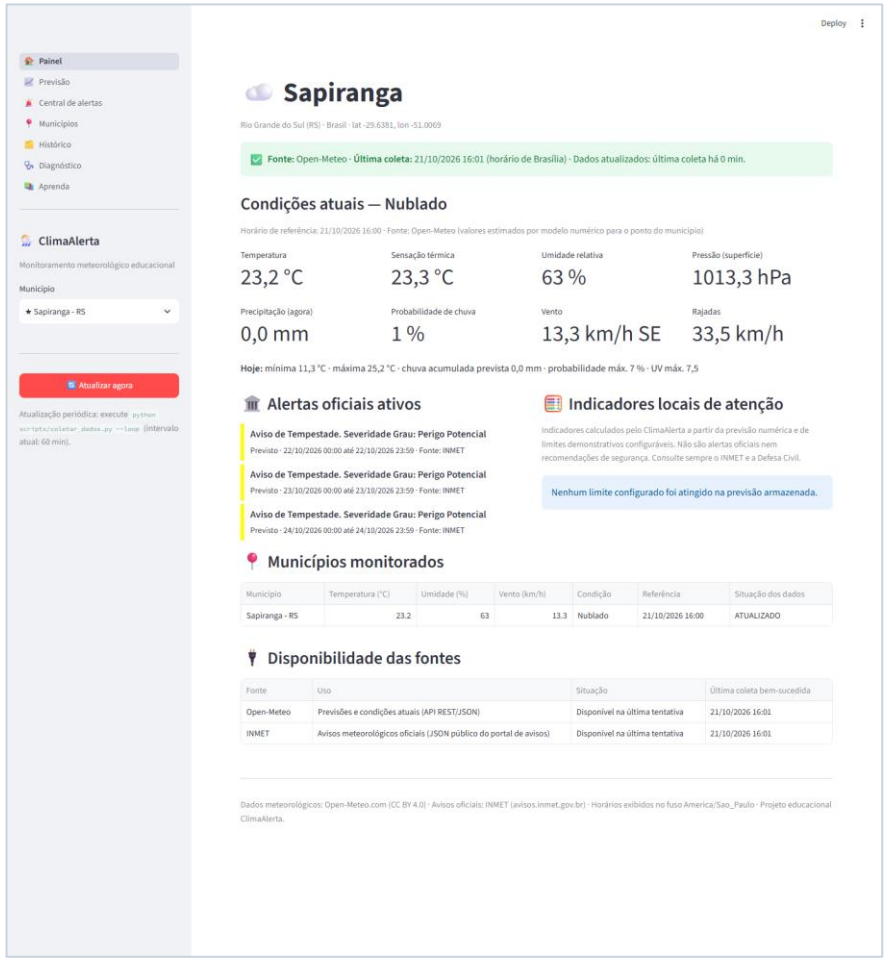
Material de aula — acompanha o manual pedagógico (PDF)
Versão 1.1 (PyCharm) · 21/10/2026

O que vamos construir

- Um sistema web local que mostra **condições atuais**, previsão de **24 h** e **7 dias**
- **Avisos oficiais** do INMET separados de **indicadores calculados** por nós
- Vários municípios — começando por **Sapiranga - RS**
- Histórico, exportação CSV, diagnóstico e página educacional
- Web Scraping com **Selenium** em uma página de demonstração
- Testes automatizados, depuração e documentação técnica completa

> Projeto educacional: indicadores NÃO são alertas oficiais. Em risco, siga o INMET e a Defesa Civil (199).

O resultado final



Painel do ClimaAlerta com dados reais (21/10/2026)

- Fonte e horário sempre visíveis
- Alertas oficiais × indicadores
- Funciona sem internet com os últimos dados salvos

Os 15 módulos

Módulos	Tema	Você cria
1–2	PyCharm, ambiente e dependências; fundamentos de Python	m01, m02
3–4	Primeira dashboard; HTML e seletores CSS	m03, m04
5–6	Selenium e depurador; consumo de APIs	m05, m06
7–8	Pandas; SQLite e SQLAlchemy	m07, m08
9–10	Regras de negócio; interface	m09, m10
11–12	Integração; automação	estudo do sistema, m12
13–15	Testes e depuração; Git; projeto final	test_m09, commits, IND08

Como cada módulo funciona

- **Objetivo → conceitos → arquivo → passo a passo no PyCharm → código**
- Código **completo e comentado** linha a linha no manual
- **Resultado esperado**: saída real, obtida executando o código
- **Verificação**: um teste automático confere o seu arquivo (▶ ao lado do teste)
- **Erros frequentes**, **exercícios**, **critérios de conclusão** e **desafios**

> Cada ação no PyCharm tem o comando equivalente no Terminal do PyCharm (Alt+F12).

Módulo 1

Preparação do ambiente

PyCharm, Python, ambiente virtual e instalação das dependências

Conhecendo a janela do PyCharm

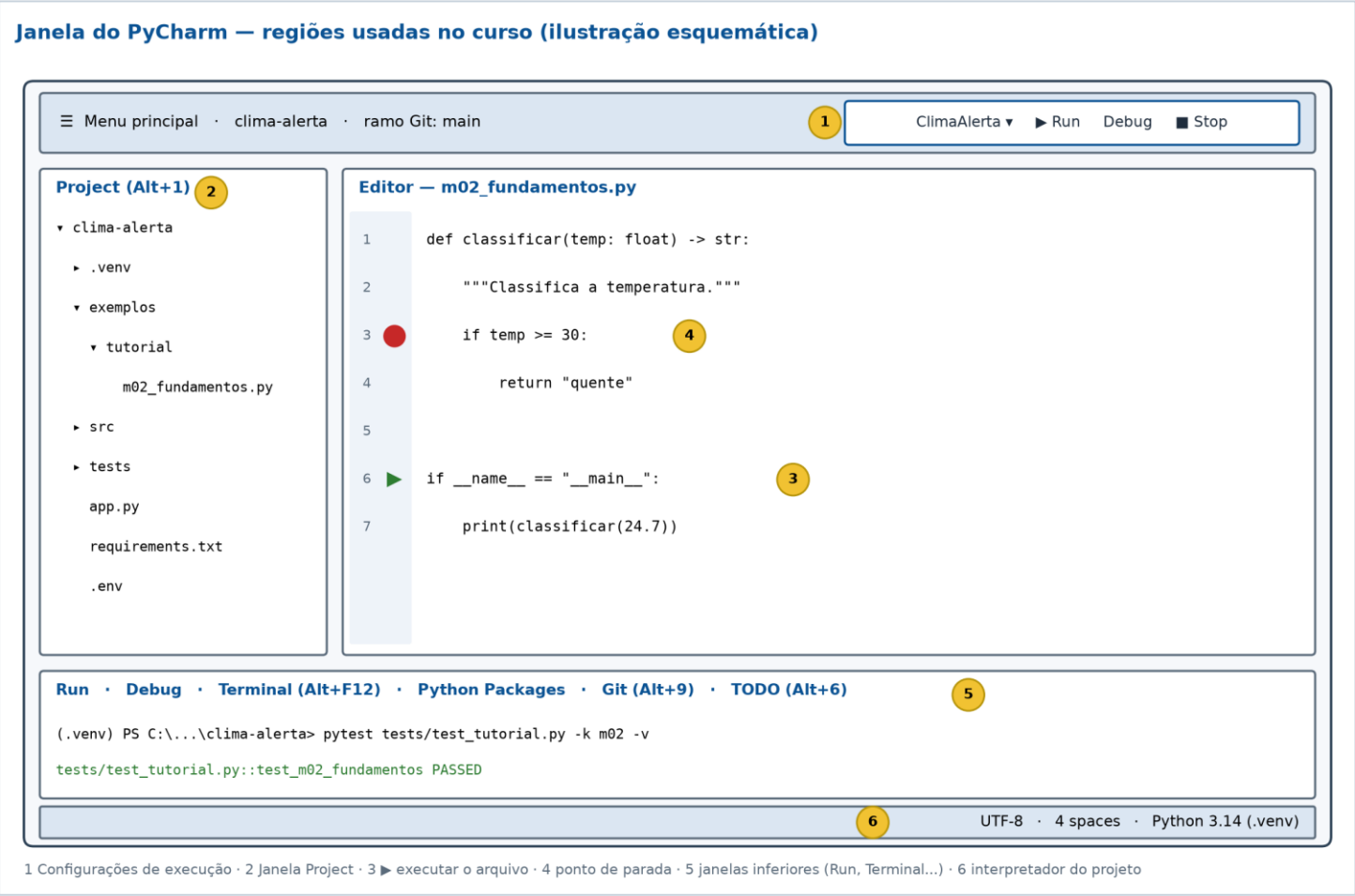


Ilustração esquemática (não é captura de tela)

Atalhos essenciais

Ação	Atalho
Executar o arquivo aberto	Ctrl+Shift+F10
Executar / depurar a última configuração	Shift+F10 / Shift+F9
Parar	Ctrl+F2
Terminal / janela Project	Alt+F12 / Alt+1
Settings	Ctrl+Alt+S
Procurar qualquer coisa	Shift duas vezes
Documentação / ir para a definição	Ctrl+Q / Ctrl+B
Reformatar código / sugestões	Ctrl+Alt+L / Alt+Enter

Abrir o projeto e criar o ambiente virtual

- **Open** › pasta raiz (a que contém `app.py`) › **Trust Project**
- Settings (Ctrl+Alt+S) › Project › **Python Interpreter**
  - Add Interpreter › Add Local Interpreter
  - Generate new · **Virtualenv** · Location `<raiz>/.venv`
- Barra de status mostra `Python 3.x (.venv)`
- Se o `.venv` já existir: **Select existing** › `.venv/Scripts/python.exe`

> O PyCharm pode oferecer criar o ambiente sozinho: aceite e confira que a pasta é .venv na raiz.

Instalando as dependências — três caminhos

Caminho	Como fazer no PyCharm
A — Install requirements (recomendado)	Abra `requirements.txt` e clique em **Install requirements** na faixa do topo do editor
B — Python Packages	View › Tool Windows › **Python Packages**: pesquise o pacote, escolha a versão e **Install**
C — Terminal do PyCharm	`python -m pip install -r requirements.txt`
Conferir	Settings › Python Interpreter: lista de pacotes e versões

> Ao acrescentar uma biblioteca no requirements.txt, o PyCharm volta a oferecer Install requirements.

Ajustes do PyCharm (uma vez por projeto)

Ajuste	Onde
UTF-8	Settings › Editor › File Encodings
pytest	Settings › Tools › Python Integrated Tools › Default test runner
Pasta de trabalho = raiz	Run › Edit Configurations › Edit configuration templates › Working directory: `$ProjectFileDir$`
Pasta do tutorial	Renomear soluções para `tutorial_referencia`; criar `exemplos/tutorial`; Mark Directory as › Sources Root
.env	Copiar `.env.example` como `.env` (Ctrl+C / Ctrl+V)

Primeiro programa: verificar o ambiente

▶ m01_verificar_ambiente.py — janela Run

```
Python 3.14.3 em ...\.venv\Scripts\python.exe
  versão do Python: OK
  ambiente virtual ativo: sim
  requests        2.34.2
  pandas          3.0.6
  streamlit       1.66.1
  selenium        4.51.0
  SQLAlchemy      2.1.6
Ambiente pronto!
```

Como verificar

- New › Python File: `m01_verificar_ambiente`
- ▶ ao lado de `if __name__ ...`
- Teste: ▶ em `test_m01_verifica_ambiente`

Exercícios — Módulo 1

- Em Settings › Python Interpreter, anote as versões de `pandas` e `selenium`
- Na janela Python Packages, pesquise `ruff` e leia a descrição
- Abra o `.env`: qual é o intervalo de atualização padrão?

DESAFIO DE EXTENSÃO

Faça o verificador terminar com sys.exit(1) quando algo estiver errado e veja o exit code na janela Run.

Módulo 2

Fundamentos de Python

Variáveis, listas, dicionários, funções, condições e laços

Conceitos

Dados

- `str`, `int`, `float`, `bool`
- Lista `[11.3, 14.0]`
- Dicionário `{"cidade": ...}`
- Decimal com **ponto**: `24.7`

Lógica

- `def` cria funções; `return` devolve
- `if / elif / else`
- `for` percorre listas
- f-string: `f"{temp:.1f} °C"`

Funções do módulo 2

exemplos/tutorial/m02_fundamentos.py (trecho sem comentários — código completo e comentado no manual)

```
def celsius_para_fahrenheit(celsius: float) -> float:
    return celsius * 9 / 5 + 32
def classificar(temp: float) -> str:
    if temp >= 30:
        return "quente"
    elif temp >= 18:
        return "agradável"
    else:
        return "frio"
def media(valores: list[float]) -> float:
    return sum(valores) / len(valores)
if __name__ == "__main__":
    print(f"Cidade: {leitura['cidade']}")
    print(f"Agora: {temperatura} °C = {celsius_para_fahrenheit(temperatura):.1f} °F ({classificar(temperatura)})")
```

A primeira condição verdadeira vence · Ctrl+Q: documentação · Ctrl+B: definição · Ctrl+Alt+L: formatar

Resultado e verificação

▶ m02_fundamentos.py — janela Run

```
Cidade: Sapiranga
Agora: 24.7 °C = 76.5 °F (agradável)
Mínima: 11.3 °C | Máxima: 25.2 °C | Média: 19.0 °C
  00h ->  11.3 °C (frio)
  08h ->  19.5 °C (agradável)
Está chovendo? não
```

Como verificar

- ▶ em `test_m02_fundamentos`
- `IndentationError`? use Tab e Ctrl+Alt+L
- Shift+F6 renomeia em todo o arquivo

Exercícios — Módulo 2

- Crie `kmh_para_ms(v)` e converta 13,5 km/h
- Acrescente a faixa "muito quente" (≥ 35 °C)
- Conte as horas classificadas como "frio"

DESAFIO DE EXTENSÃO

Leia a temperatura com input() (digitação na janela Run) e aceite vírgula decimal.

Módulo 3

Primeira dashboard com Streamlit

Cartões, seletor, gráfico, tabela e Run Configuration

Conceitos

- **Dashboard**: cartões, gráficos e tabelas que resumem dados
- O Streamlit **reexecuta o script** a cada interação
- Uma dashboard é iniciada com `streamlit run arquivo.py` — **não** com o ▶ comum
- No PyCharm: Run Configuration do **módulo** `streamlit`
- Dados **simulados** sempre identificados na tela

Run Configuration para o Streamlit

Campo (Run › Edit Configurations › + › Python)	Valor
Name	`m03 - dashboard`
script → **module**	`streamlit`
Script parameters	`run exemplos/tutorial/m03_primeira_dashboard.py`
Working directory	pasta raiz do projeto
Python interpreter	`.venv` do projeto

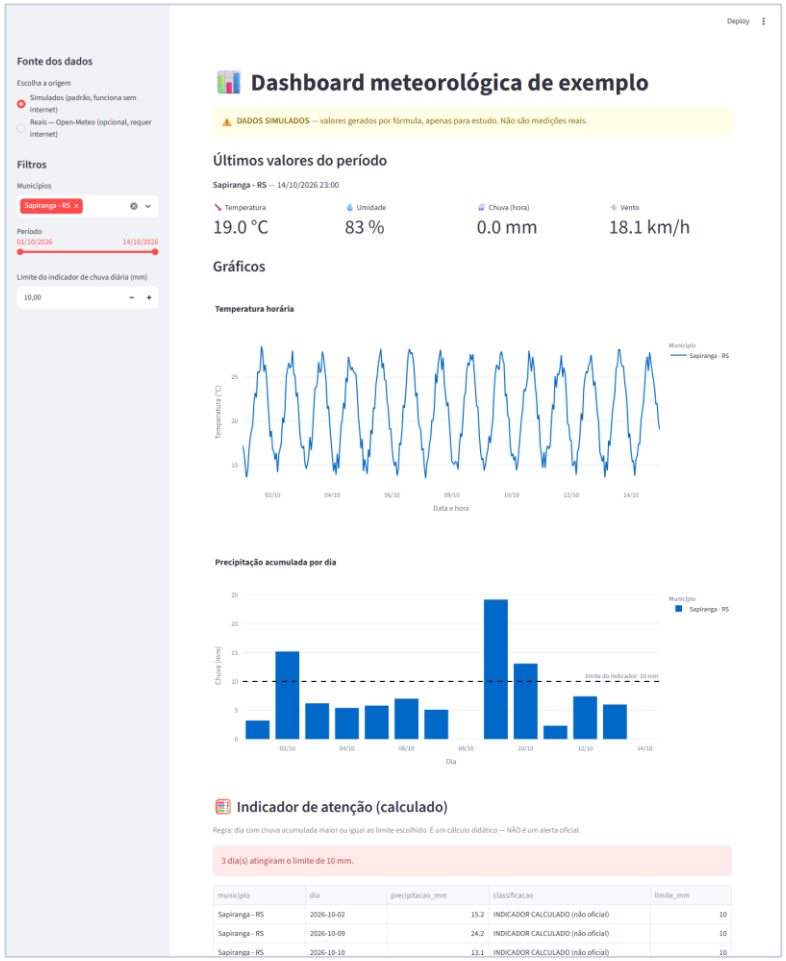
> Para parar: ■ na janela Run (Ctrl+F2). Copy Configuration duplica a configuração para outras dashboards.

Cartões e gráfico

exemplos/tutorial/m03_primeira_dashboard.py (trecho sem comentários — código completo e comentado no manual)

```
col1, col2, col3 = st.columns(3)
col1.metric("Máxima", f"{dados['temperatura'].max():.1f} °C")
col2.metric("Mínima", f"{dados['temperatura'].min():.1f} °C")
col3.metric("Chuva total", f"{dados['chuva_mm'].sum():.1f} mm")
variavel = st.selectbox("Variável do gráfico", ["temperatura", "chuva_mm"])
st.line_chart(dados, x="hora", y=variavel)
st.subheader("Tabela")
st.dataframe(dados, hide_index=True)
```

Dashboard de exemplo do projeto



Configuração 'Dashboard de exemplo' — run exemplos/dashboard_python/app.py

- Filtros de município e período
- Indicador calculado
- Dados reais opcionais (Open-Meteo)
- Nunca mistura simulados e reais

Exercícios — Módulo 3

- Acrescente um cartão com a média
- Troque `line_chart` por `bar_chart`
- Na dashboard de exemplo, use limite de 5 mm: quantos dias aparecem?

DESAFIO DE EXTENSÃO

Leia dados_simulados.csv com pd.read_csv e mostre-o na sua página.

Módulo 4

HTML e seletores CSS

A base do Web Scraping

Seletores CSS essenciais

Seletor	Encontra	Na página do módulo
`h1`	Pela tag	O título
`#titulo`	Pelo id (único)	O título
`.cidade`	Pela classe	3 cartões
`div.cidade .temp`	Dentro de outro	3 temperaturas
`.cidade.destaque`	Duas classes	Porto Alegre
`[data-uf='SC']`	Pelo atributo	Florianópolis

A página de prática

exemplos/tutorial/m04_pagina.html (trecho — arquivo completo e comentado no manual)

```
<div class="cidade" data-uf="RS">
  <h2 class="nome">Sapiranga</h2>                              <!-- seletor: .nome -->
  <span class="temp">24,5 °C</span>                             <!-- seletor: .temp (vírgula decimal!) -->
  <a class="mais" href="https://open-meteo.com/">Fonte de exemplo</a> <!-- link: atributo href -->
</div>

<!-- Este cartão tem DUAS classes: "cidade" e "destaque" (seletor: .cidade.destaque) -->
<div class="cidade destaque" data-uf="RS">
  <h2 class="nome">Porto Alegre</h2>
  <span class="temp">26,1 °C</span>
  <!-- Repare: este cartão NÃO tem o link <a class="mais">.
       No módulo 5 o programa precisa tratar esse elemento ausente. -->
```

New › HTML File · Ícone do Chrome no canto do editor · F12 › Console · `document.querySelectorAll(".cidade").length` → **3**

Exercícios — Módulo 4

- Seletor para os nomes das cidades do RS
- Acrescente uma 4ª cidade e confirme no console
- Abra a página de demonstração do projeto pelo PyCharm e descubra a classe dos cartões

DESAFIO DE EXTENSÃO

Use li:nth-child(2) e escreva o XPath equivalente a #titulo.

Módulo 5

Web Scraping com Selenium

Esperas explícitas, normalização — e o depurador do PyCharm

Conceitos

- **WebDriver** controla o Chrome; `--headless=new` sem janela
- **Espera explícita**: `WebDriverWait(...).until(...)`
- `find_element` (erro se faltar) × `find_elements` (lista vazia)
- **Normalizar**: "24,5 °C" → `24.5`
- `try/finally` garante `navegador.quit()`

> Prefira APIs. Respeite termos de uso e robots.txt; nunca contorne CAPTCHA ou login.

Extração com tratamento de ausência

exemplos/tutorial/m05_selenium_basico.py (trecho sem comentários — código completo e comentado no manual)

```
        navegador.get(PAGINA)
        WebDriverWait(navegador, 10).until(EC.presence_of_element_located((By.ID, "titulo")))
        resultado = []
        for cartao in navegador.find_elements(By.CSS_SELECTOR, "div.cidade"):
            temp_texto = texto_ou_none(cartao, ".temp")
            temperatura = float(temp_texto.split()[0].replace(",", ".")) if temp_texto else None
            link = cartao.find_elements(By.CSS_SELECTOR, "a.mais")
            resultado.append(
                {
                    "cidade": texto_ou_none(cartao, ".nome"),
                    "uf": cartao.get_attribute("data-uf"),
                    "temperatura": temperatura,
                    "link": link[0].get_attribute("href") if link else None,
                }
            )
        return resultado
    finally:
        navegador.quit()
```

Acompanhando a extração no depurador

Passo	Como
Ponto de parada	Clique na margem da linha `for cartao in ...` (bolinha vermelha)
Depurar	Botão direito no editor › **Debug 'm05_selenium_basico'**
Próxima linha	**F8** (Step Over) — observe `cartao`, `temp_texto`, `temperatura`
Avaliar expressão	**Alt+F8** › `cartao.text`
Continuar até o fim	**F9** (o `finally` fecha o navegador)

Resultado e verificação

▶ m05_selenium_basico.py — janela Run

```
{'cidade': 'Sapiranga', 'uf': 'RS', 'temperatura': 24.5,
 'link': 'https://open-meteo.com/'}
{'cidade': 'Porto Alegre', 'uf': 'RS', 'temperatura': 26.1,
 'link': None}
{'cidade': 'Florianópolis', 'uf': 'SC', 'temperatura': 22.0,
 'link': None}
```

Como verificar

- ▶ em `test_m05_selenium`
- Scraper do projeto: ▶ em `scripts/executar_scraper_demo.py`
- Script parameters `--visivel` mostra o Chrome

Exercícios — Módulo 5

- Extraia os dias de `#lista-dias li` como números
- No depurador, avalie `cartao.text` com Alt+F8
- No scraper de demonstração: qual cidade não tem umidade?

DESAFIO DE EXTENSÃO

Insira as cidades com setTimeout de 2 s e ajuste a espera.

Módulo 6

Consumo de APIs

requests, parâmetros, JSON, status HTTP e timeout

Conceitos

API

- Pedido (requisição) → resposta
- **GET** com parâmetros
- Resposta em **JSON**
- Status 200 / 4xx / 5xx

Boas práticas

- Sempre `timeout=`
- `raise_for_status()`
- Tratar `RequestException`
- Citar a fonte (CC BY 4.0)

Consultando a Open-Meteo

exemplos/tutorial/m06_api_open_meteo.py (trecho sem comentários — código completo e comentado no manual)

```
    parametros = {
        "latitude": latitude,
        "longitude": longitude,
        "current": "temperature_2m,relative_humidity_2m,wind_speed_10m",
        "timezone": "America/Sao_Paulo",
    }
    resposta = http.get(URL, params=parametros, timeout=15)
    resposta.raise_for_status()
    dados = resposta.json()
    atual = dados["current"]
    return {
        "horario": atual["time"],
        "temperatura": atual["temperature_2m"],
        "umidade": atual["relative_humidity_2m"],
        "vento": atual["wind_speed_10m"],
    }
if __name__ == "__main__":
    try:
```

Resultado e verificação

▶ m06_api_open_meteo.py — janela Run

```
Condições atuais em Sapiranga (Open-Meteo):
  horario: 2026-10-21T15:45
  temperatura: 23.6
  umidade: 62
  vento: 13.5
```

Como verificar

- ▶ em `test_m06_api_com_resposta_real_gravada`
- Debug: expanda `dados` para ver o JSON
- Pacote sublinhado? Alt+Enter › Install package

Exercícios — Módulo 6

- Inclua `apparent_temperature`
- Compare Sapiranga com Porto Alegre (−30,03283; −51,23019)
- Peça a máxima e a mínima de hoje (`daily`)

DESAFIO DE EXTENSÃO

Use a API de geocodificação para descobrir as coordenadas de outra cidade.

Módulo 7

Tratamento de dados com Pandas

DataFrame, datas, filtros, agrupamentos e CSV

Resumo por dia

exemplos/tutorial/m07_pandas.py (trecho sem comentários — código completo e comentado no manual)

```
def resumo_por_dia(df: pd.DataFrame) -> pd.DataFrame:
    df = df.assign(dia=df["horario"].dt.date)
    return (
        df.groupby("dia")
        .agg(
            minima=("temperatura", "min"),
            maxima=("temperatura", "max"),
            chuva_total=("chuva_mm", "sum"),
        )
        .round(1)
        .reset_index()
    )
```

- `pd.to_datetime`: texto → data
- Filtro: `df[df["temperatura"] >= 25]`
- Debug › botão direito › **View as DataFrame**

Resultado e verificação

▶ m07_pandas.py — janela Run

```
168 linhas; temperatura média: 18.9 °C
Horas com 25 °C ou mais: 18

          dia  minima  maxima  chuva_total
0  2026-10-21    11.5    25.1          0.0
1  2026-10-22    14.8    27.5          0.9
4  2026-10-25    14.5    17.9         25.2
```

Como verificar

- ▶ em `test_m07_pandas`
- `resumo_diario.csv` aparece na janela Project
- Dados reais gravados: funciona offline

Exercícios — Módulo 7

- Umidade média por dia
- Hora mais quente da semana (`idxmax`)
- Horas com probabilidade de chuva ≥ 70 %

DESAFIO DE EXTENSÃO

Compare seu resumo com a previsão diária da API. Por que a máxima difere?

Módulo 8

SQLite e SQLAlchemy

Tabelas, chaves, restrições e consultas

Conceitos

Modelagem

- Tabela = classe; linha = objeto
- **PK**: identifica a linha
- **FK**: aponta outra tabela
- Relacionamento **1:N**

Integridade

- **UNIQUE** evita duplicação
- `commit` grava; `rollback` desfaz
- Idempotente: rodar 2× não duplica
- SQLite = um arquivo

Modelo com restrição

exemplos/tutorial/m08_banco.py (trecho sem comentários — código completo e comentado no manual)

```
class Observacao(Base):
    __tablename__ = "observacao"
    __table_args__ = (UniqueConstraint("municipio_id", "horario"),)
    id: Mapped[int] = mapped_column(primary_key=True)
    municipio_id: Mapped[int] = mapped_column(ForeignKey("municipio.id"))
    horario: Mapped[datetime]
    temperatura: Mapped[float | None] = mapped_column(Float)
    municipio: Mapped[Municipio] = relationship(back_populates="observacoes")
def preparar(caminho: Path = BANCO):
    engine = create_engine(f"sqlite:///{caminho.as_posix()}")
```

Mesma hora + mesmo município = recusado · Ver o banco: DB Browser for SQLite ou janela Database (Pro)

Resultado e verificação

▶ m08_banco.py (2 vezes) — janela Run

```
Sapiranga 21/10 12:00 -> 24.4 °C
Sapiranga 21/10 15:00 -> 24.5 °C
Sapiranga 21/10 18:00 -> 20.2 °C
Duplicar 21/10 09:00: recusado pela restrição UNIQUE
```

Como verificar

- ▶ em `test_m08_banco`
- Botão direito em `tests/test_database.py` › Run
- Mudou o modelo? apague `tutorial.db`

Exercícios — Módulo 8

- Acrescente a coluna `umidade`
- Média por município com `func.avg`
- Abra `tutorial.db` em um visualizador de SQLite

DESAFIO DE EXTENSÃO

Estude o upsert (INSERT … ON CONFLICT DO UPDATE) em src/database/repositories.py.

Módulo 9

Regras de negócio

Do texto em português para o código

Alerta oficial × indicador calculado

Alerta oficial (INMET)

- Emitido por autoridade
- Severidade padronizada
- Link para a publicação
- Gravado como OFICIAL

Indicador local (nosso)

- Cálculo com limite configurável
- Limites **demonstrativos**
- Sempre "não oficial"
- Calculado na hora; nunca gravado como alerta

Indicador de rajada

exemplos/tutorial/m09_regras.py (trecho sem comentários — código completo e comentado no manual)

```
    validos = [p for p in previsao if p.get("rajada") is not None and p["rajada"] >= limite]
    if not validos:
        return None
    pior = max(validos, key=lambda p: p["rajada"])
    return {
        "condicao": "Rajadas de vento acima do limite",
        "valor": pior["rajada"],
        "limite": limite,
        "horario": pior["horario"],
        "ocorrencias": len(validos),
        "classificacao": "INDICADOR CALCULADO (não oficial)",
    }
if __name__ == "__main__":
    previsao = [
```

Ausente (None) nunca dispara · Limite inclusivo (>=) · Experimente no Python Console

Exercícios — Módulo 9

- Escreva `indicador_chuva(previsao, limite=10)`
- Limite de 50 km/h: quantas ocorrências?
- Escreva em português a regra "umidade baixa"

DESAFIO DE EXTENSÃO

Leia o limite de uma variável de ambiente (Run Configuration › Environment variables).

Módulo 10

Desenvolvimento da interface

A interface lê o banco — e não a internet

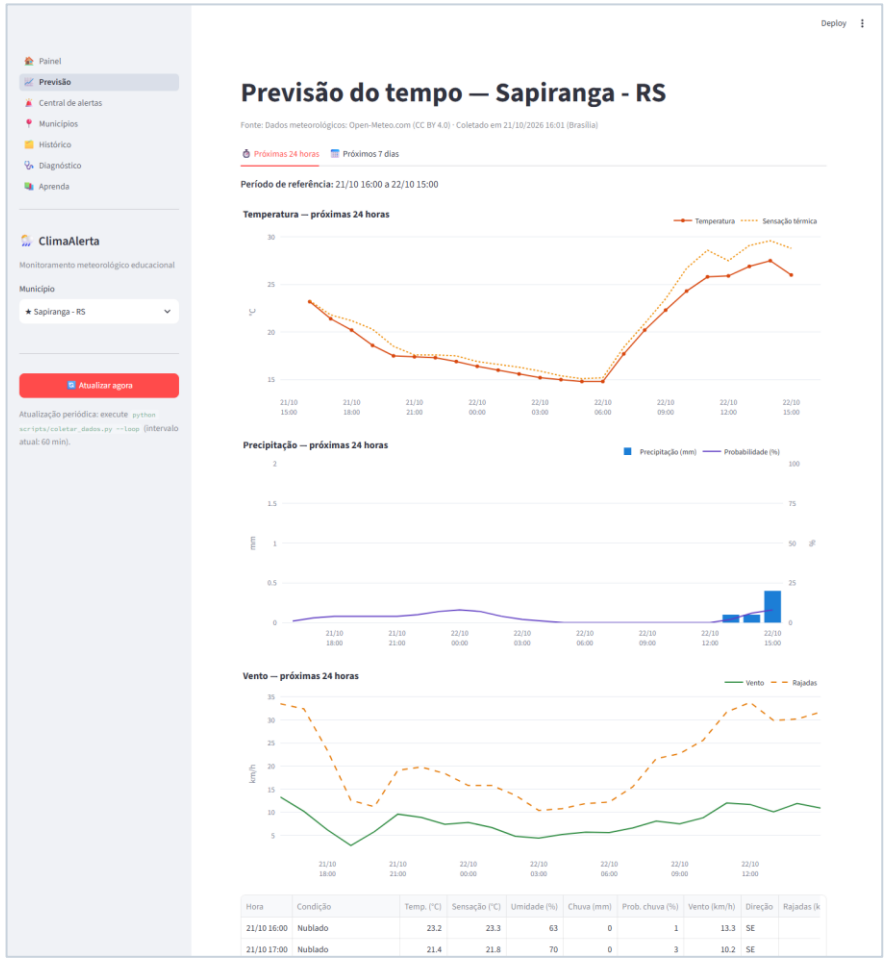
Interface lendo o banco

exemplos/tutorial/m10_interface.py (trecho sem comentários — código completo e comentado no manual)

```
with Session(motor) as sessao:
    consulta = select(Municipio.nome, Observacao.horario, Observacao.temperatura).join(Observacao)
    df = pd.DataFrame(sessao.execute(consulta).all(), columns=["municipio", "horario", "temperatura"])
motor.dispose()
if df.empty:
    st.info("O banco está vazio. Execute primeiro: python exemplos/tutorial/m08_banco.py")
    st.stop()
municipio = st.selectbox("Município", sorted(df["municipio"].unique()))
minima = st.slider("Mostrar temperaturas a partir de (°C)", 0, 40, 0)
filtrado = df[(df["municipio"] == municipio) & (df["temperatura"] >= minima)]
st.metric("Registros exibidos", len(filtrado))
grafico = px.line(filtrado, x="horario", y="temperatura", markers=True, title=f"Temperatura — {municipio}")
st.plotly_chart(grafico)
st.dataframe(filtrado, hide_index=True)
```

Antes: ▶ `m08_banco.py` · Configuração `m10 - interface` (module streamlit) · `m08_banco` sublinhado? Sources Root

A interface completa



Configuração 'ClimaAlerta' — module streamlit, parâmetros run app.py

- 7 páginas com `st.navigation`
- Unidades em todos os números
- Fonte e horário dos dados

Exercícios — Módulo 10

- Cartão com a temperatura máxima do filtro
- Troque por `px.bar` e compare
- Qual página do ClimaAlerta mostra a última coleta de cada fonte?

DESAFIO DE EXTENSÃO

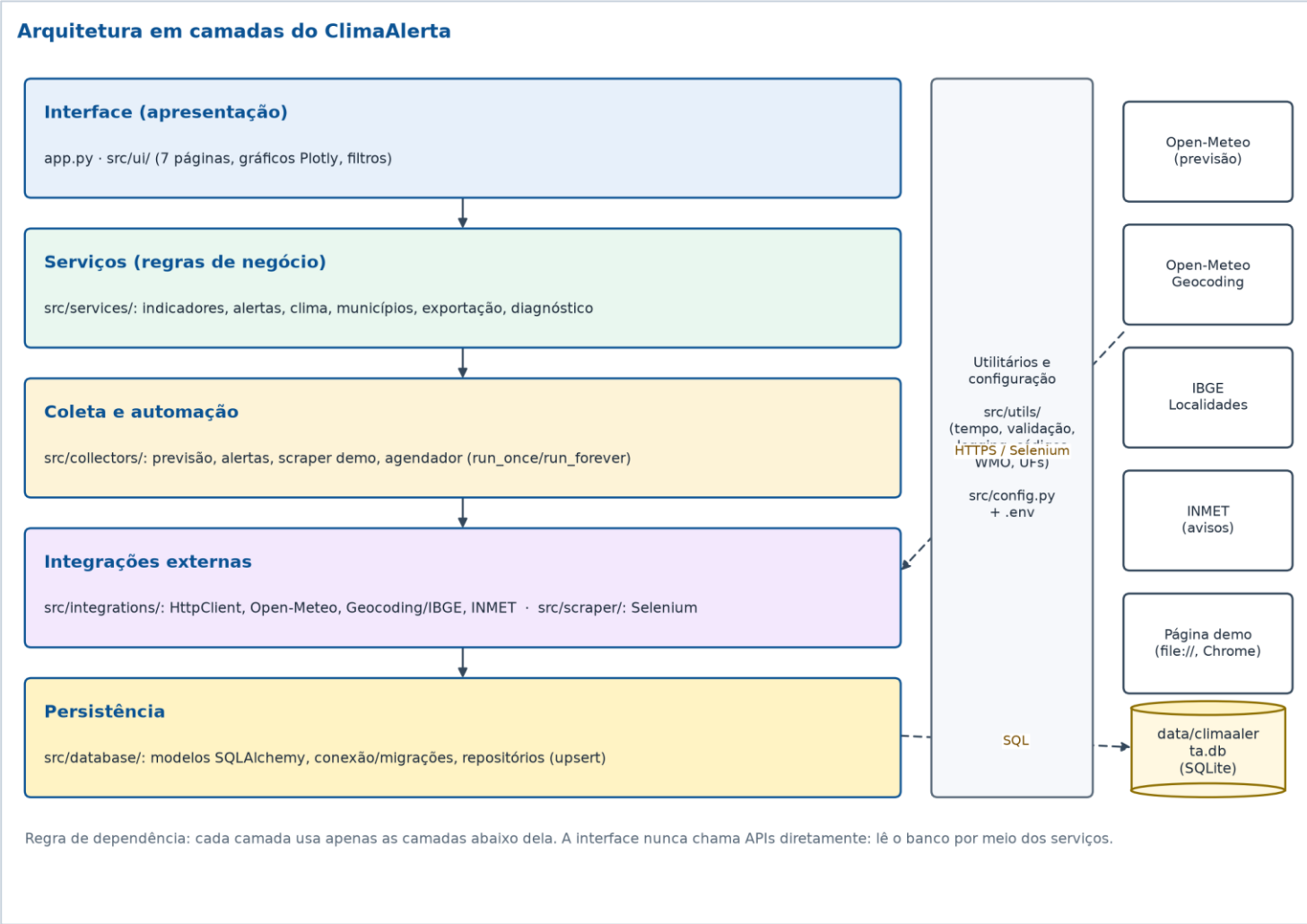
Adicione st.download_button para exportar o filtro em CSV.

Módulo 11

Integração dos módulos

O ClimaAlerta completo com dados reais

Arquitetura em camadas



Executando e navegando pelo sistema no PyCharm

Ação	Como
Criar o banco	▶ em `scripts/inicializar_banco.py`
Coletar dados reais	▶ em `scripts/coletar_dados.py`
Abrir a interface	Configuração `ClimaAlerta` ▶
Ir para a definição / onde é usado	Ctrl+B / Alt+F7
Estrutura do arquivo / abrir arquivo	Ctrl+F12 / Ctrl+Shift+N
Tarefas pendentes (esqueleto)	Janela **TODO** (Alt+6)

> Saída real: INMET 10 registros; Open-Meteo 176 registros (1 observação + 168 horas + 7 dias).

O que você fez × o sistema completo

Seu arquivo	No ClimaAlerta
m06 `buscar_condicoes`	`src/integrations/open_meteo.py`
m08 `popular`	`src/database/repositories.py` (upsert)
m09 `indicador_rajada`	`src/services/indicators.py` (7 regras)
m10 interface	`src/ui/page_*.py`
m05 scraper	`src/scraper/`

Exercícios — Módulo 11

- Mude `LIMITE_PROBABILIDADE_CHUVA` para 5, reinicie e observe; depois volte para 70
- Cadastre outro município e compare no Painel
- Use Alt+F7 em `evaluate_indicators`

DESAFIO DE EXTENSÃO

Explique como o filtro por município usa o código IBGE (alert_service.py).

Módulo 12

Atualização automatizada

Laços temporizados, robustez e argumentos

Laço robusto

exemplos/tutorial/m12_agendador.py (trecho sem comentários — código completo e comentado no manual)

```
def executar(vezes: int, intervalo: float, coletar=buscar_condicoes, esperar=time.sleep) -> list[dict]:
    resultados = []
    for rodada in range(1, vezes + 1):
        try:
            dados = coletar(-29.63806, -51.00694)
            resultados.append(dados)
            print(f"[{datetime.now():%H:%M:%S}] rodada {rodada}: {dados['temperatura']} °C")
        except Exception as erro:
            print(f"[{datetime.now():%H:%M:%S}] rodada {rodada}: falha ({erro})")
        if rodada < vezes:
            esperar(intervalo)
    return resultados
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Coleta repetida (tutorial)")
```

Argumentos no PyCharm

Configuração	Script parameters
`m12_agendador`	`--vezes 3 --intervalo 5`
`coletar_dados --loop`	`--loop --intervalo 15`
Parar	■ na janela Run (Ctrl+F2)
Saída real	`[15:45:44] rodada 1: 23.6 °C · [15:45:50] rodada 2: 23.6 °C`

> Execute uma vez com ▶ para o PyCharm criar a configuração; depois edite os parâmetros.

Exercícios — Módulo 12

- Desligue o Wi-Fi em uma rodada e observe
- Mostre também a umidade
- Deixe `coletar_dados --loop` rodando e veja o Histórico crescer

DESAFIO DE EXTENSÃO

Grave cada rodada em um CSV com data e hora.

Módulo 13

Testes e depuração no PyCharm

pytest, árvore de testes, tracebacks e depurador

Seus primeiros testes

exemplos/tutorial/test_m09_regras.py (trecho sem comentários — código completo e comentado no manual)

```
def test_dispara_com_o_maior_valor():
    previsao = [{"horario": "a", "rajada": 61}, {"horario": "b", "rajada": 80}]
    resultado = indicador_rajada(previsao)
    assert resultado["valor"] == 80
    assert resultado["ocorrencias"] == 2
def test_limite_e_inclusivo():
    assert indicador_rajada([{"horario": "a", "rajada": 60}]) is not None
def test_nao_dispara_abaixo_do_limite():
    assert indicador_rajada([{"horario": "a", "rajada": 59.9}]) is None
def test_valor_ausente_nunca_dispara():
    assert indicador_rajada([{"horario": "a", "rajada": None}]) is None
def test_classificacao_nao_oficial():
    resultado = indicador_rajada([{"horario": "a", "rajada": 90}])
    assert "não oficial" in resultado["classificacao"]
```

▶ ao lado de cada teste · Árvore: ✓ verde / ✗ vermelho · Rerun Failed Tests

Depurador do PyCharm

Ação	Atalho
Ponto de parada	Clique na margem ou Ctrl+F8
Depurar um teste	Botão direito no ▶ do teste › Debug
Próxima linha / entrar na função / sair	F8 / F7 / Shift+F8
Continuar	F9
Avaliar expressão	Alt+F8

> Teste do teste: troque >= por > e veja test_limite_e_inclusivo falhar. Desfaça com Ctrl+Z.

Suíte completa e qualidade

Terminal do PyCharm (Alt+F12) — pasta raiz

```
pytest exemplos/tutorial/test_m09_regras.py -v
pytest -v
pytest --cov=src --cov-report=term
ruff check .
```

> No PyCharm: botão direito na pasta tests › Run 'pytest in tests'; Code › Inspect Code.

Exercícios — Módulo 13

- Teste para `indicador_chuva`
- Teste `celsius_para_fahrenheit(0) == 32`
- Provoque um erro e clique no link do traceback na janela Run

DESAFIO DE EXTENSÃO

Use @pytest.mark.parametrize; estude o FakeHttp em tests/conftest.py.

Módulo 14

Git e documentação no PyCharm

Histórico de versões e documentos que acompanham o código

Git integrado ao PyCharm

Ação	Como
Ativar o Git	VCS › Enable Version Control Integration › Git
Commit	Ctrl+K — marque os arquivos e escreva a mensagem
Histórico / diferenças	Alt+9 › Log · botão direito › Git › Show Diff
Novo ramo / mesclar	Widget de ramos no topo › New Branch · Git › Merge
Publicar no GitHub	Git › GitHub › **Share Project on GitHub**
Enviar / receber	Ctrl+Shift+K (Push) / Ctrl+T (Update)

> Configure uma vez no Terminal: git config --global user.name e user.email.

Documentos do projeto

- **01–02** Requisitos e regras de negócio
- **03–04** Arquitetura e dicionário de dados
- **05–07** BPMN, fluxogramas e casos de uso
- **08** Plano e relatório de testes (resultados reais)
- **09–10** Implantação, segurança e limitações
- **11–12** Matriz de rastreabilidade e manual técnico

> Os PDFs são gerados a partir do código, do banco e do resultado real dos testes.

Exercícios — Módulo 14

- Faça 3 commits com mensagens claras e veja o Log
- Encontre na matriz o teste do RF014
- Abra com Ctrl+Shift+N o módulo de um requisito do documento 01

DESAFIO DE EXTENSÃO

Crie um ramo por exercício e una-os com Merge.

Módulo 15

Projeto final e apresentação

IND08 — Índice UV: código, testes, configuração e documentação

Projeto final: o que alterar

Arquivo	Alteração (código completo no manual)
`src/config.py`	Campo `uv_index_max` em `IndicatorThresholds`; leitura de `LIMITE_UV_MAXIMO` em `get_settings`
`src/services/indicators.py`	Regra IND08 em `build_rules`
`.env.example` e `.env`	`LIMITE_UV_MAXIMO=8`
`tests/test_indicators.py`	8 regras no teste existente + `test_uv_index_rule`

> O código completo do manual foi aplicado a uma cópia do projeto e a suíte passou (tests/test_projeto_final.py).

Teste novo do IND08

tests/test_indicators.py (função completa no manual, com comentários)

```
def test_uv_index_rule():
    rule = next(r for r in build_rules(IndicatorThresholds()) if r.rule_id == "IND08")
    hits = evaluate_daily(rule, [day(21, uv_index_max=9.1), day(22, uv_index_max=6.0)])
    assert [h.reference_time for h in hits] == [date(2026, 10, 21)]
    assert hits[0].value == 9.1 and hits[0].classification.startswith("INDICADOR CALCULADO")
```

Roteiro de apresentação (10–15 min)

Tempo	Conteúdo
1 min	Problema e objetivo
4 min	Demonstração pelo PyCharm: Painel, Previsão, Central de alertas (oficial × calculado)
2 min	Arquitetura e fontes de dados
2 min	Selenium e por que o INMET é consumido via API
2 min	Árvore de testes verde e matriz de rastreabilidade
2 min	A extensão do grupo
1 min	Limitações e próximos passos

Revisão: conceitos e recursos do PyCharm

Módulo	Conceito-chave	Recurso do PyCharm
1–2	Ambiente virtual; fundamentos	Interpreter, Install requirements
3–4	Dashboards; seletores CSS	Run Configuration de módulo
5–6	Selenium; APIs	Depurador (F8, Alt+F8)
7–8	Pandas; banco de dados	View as DataFrame
9–10	Regras; interface	Python Console, Sources Root
11–12	Integração; automação	Ctrl+B, Alt+F7, Script parameters
13–15	Testes; Git; definição de pronto	Árvore de testes, Commit, Merge

Checklist final

- ☐ Interpretador `.venv` e dependências instalados pelo PyCharm
- ☐ Arquivos m01 a m12 e test_m09 criados e testados
- ☐ Coleta real com SUCESSO nas duas fontes
- ☐ IND08 implementado com testes aprovados
- ☐ `ruff check .` sem erros e suíte completa aprovada
- ☐ Repositório Git com histórico (e no GitHub)
- ☐ Apresentação explica alerta oficial × indicador

Referências

- PyCharm: jetbrains.com/help/pycharm
- Python: docs.python.org/pt-br/3 · Streamlit: docs.streamlit.io
- Selenium: selenium.dev/documentation · Requests: requests.readthedocs.io
- Pandas: pandas.pydata.org · SQLAlchemy: docs.sqlalchemy.org
- pytest: docs.pytest.org · Git: git-scm.com/book/pt-br
- Open-Meteo: open-meteo.com/en/docs · INMET: avisos.inmet.gov.br · IBGE: servicodados.ibge.gov.br

> Manual completo: documentacao/pedagogica/manual_pedagogico_climaalerta.pdf